PROJECT PHASE 2

Cloud Dashboard Development

Diet Analysis using Azure Functions, Azure Blob Storage, and Azure Static Web Apps

Team	Anagha Roy, Jasmeen, and Manya Sethi
Member C	Manya Sethi - Integration and Documentation
Course	[Course / Section]
Instructor	[Instructor Name]
Prepared	July 17, 2026

PUBLIC FRONTEND URL

[PASTE AZURE STATIC WEB APP URL AFTER DEPLOYMENT]

GitHub Repository: github.com/anagha-04-roy/diet-analysis-phase2

1. Executive Summary

Phase 2 moves the diet analysis solution from local Azurite and localhost execution to a cloud-native deployment. The final system stores All_Diets.csv in Azure Blob Storage, processes the dataset through three HTTP-triggered Azure Functions, and presents the returned JSON in a public dashboard hosted on Azure Static Web Apps.

Member C completed the frontend-to-backend integration, added filter and refresh behavior, measured function/API response time, prepared CORS and authentication guidance, created the end-to-end test plan, assembled the repository structure, and produced the architecture and submission documentation.

Final result

The dashboard connects to GetNutritionalInsights, GetRecipes, and GetClusters; displays four distinct visualizations; provides diet, cuisine, and search controls; shows execution metadata; and includes deployment-ready static frontend files.

2. Team Work Division

Member	Assigned responsibility	Completed work
Anagha Roy - Member A	Cloud Backend and Storage	Resource group, Storage Account, Function App, live blob dataset, three deployed endpoints, endpoint testing.
Jasmeen - Member B	Frontend Dashboard	Initial dashboard interface and chart components.
Manya Sethi - Member C	Integration and Documentation	API wiring, filters, refresh, timing metadata, CORS/auth handling, end-to-end tests, architecture diagram, documentation, repository assembly, deployment checklist.

3. Solution Architecture

The architecture separates storage, serverless compute, and frontend hosting. The browser never downloads the CSV directly. Instead, each Azure Function retrieves the live blob through the Azure Storage SDK, processes it with Pandas, and returns JSON to the dashboard.

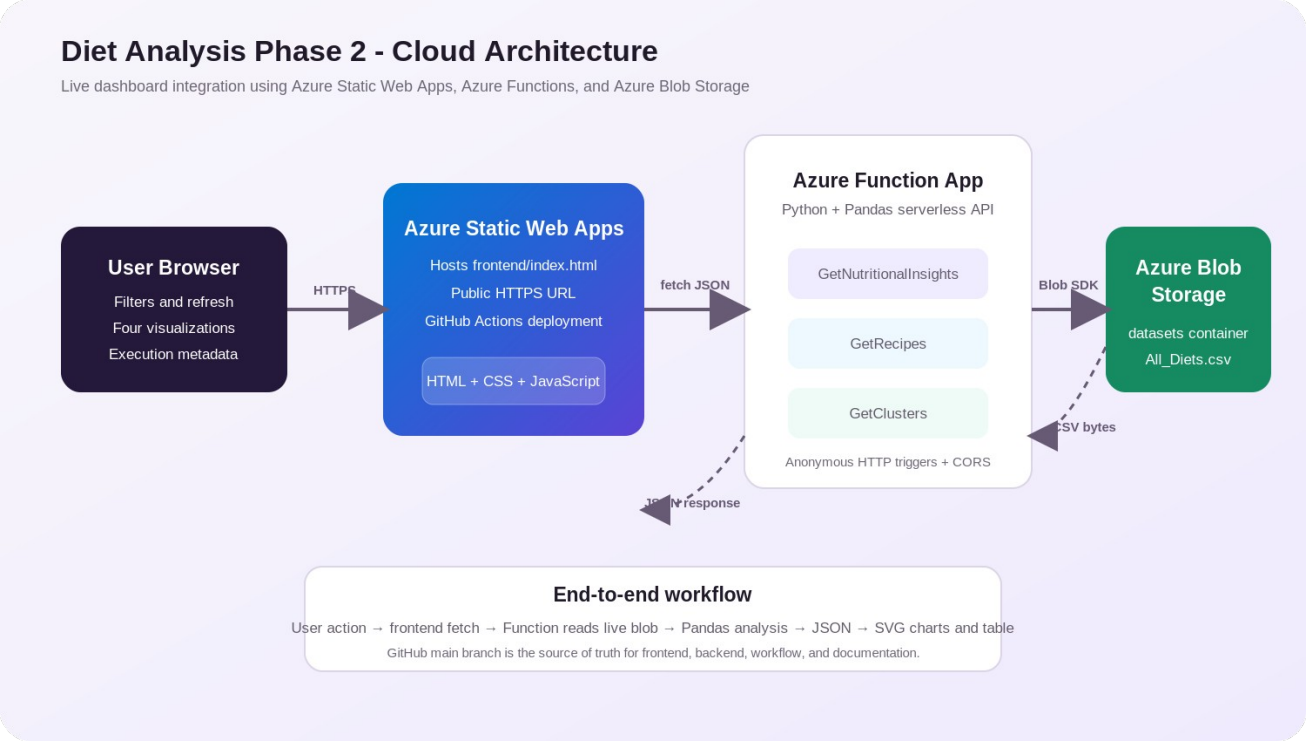

Figure 1. End-to-end Azure cloud architecture.

4. Azure Services and Responsibilities

Service	Role in solution	Cloud-native evidence
Azure Blob Storage	Stores datasets/All_Diets.csv.	Functions read the live cloud blob instead of Azurite.
Azure Function App	Runs Python/Pandas analysis through HTTP endpoints.	Serverless deployment with environment-based AzureWebJobsStorage.
Azure Static Web Apps	Hosts the public dashboard over HTTPS.	GitHub-connected deployment and publicly accessible URL.
GitHub	Stores frontend, backend, tests, workflow, and report.	Main branch becomes the final source of truth.

5. Deployed Backend Endpoints

The three endpoint URLs were verified as publicly reachable on July 17, 2026. The frontend accepts both the current array response format and a future wrapped response containing data and metadata.

Endpoint	Purpose	Dashboard use
GetNutritionalInsights	Average protein, carbohydrates, and fat grouped by diet type.	Grouped bar chart and line chart.
GetRecipes	Recipe rows with optional diet_type query filter.	Scatter plot, recipe table, search, CSV download.

Endpoint	Purpose	Dashboard use
GetClusters	Cuisine-level recipe counts and macro averages.	Cuisine distribution pie chart and cuisine filter.

Endpoint base URL

https://diet-analysis-func01-d7fdd4awc3dacqew.canadacentral-01.azurewebsites.net/api

6. Frontend Integration

The final frontend is a dependency-free static application containing HTML, CSS, JavaScript, and inline SVG chart rendering. This design minimizes Azure Static Web Apps build failures and satisfies the assignment allowance for plain JavaScript and chart-based visualization.

- All three endpoints are requested in parallel during initial load and refresh.
- The diet filter re-calls GetRecipes with a diet_type query parameter.
- Cuisine and text filters update the displayed recipes without a full page reload.
- The dashboard measures each API request with performance.now() and displays milliseconds.
- The code also reads backend execution_time_ms metadata if the Function App later returns it.
- Errors are shown clearly, including a targeted CORS troubleshooting message.
- Demo mode is available with ?demo=1 for classroom presentation or visual testing.

7. Dashboard Visualizations and Interactions

Visualization / control	Data source	Meaning
Grouped bar chart	GetNutritionalInsights	Compares average protein, carbs, and fat by diet.
Line chart	GetNutritionalInsights	Shows macro trends across diet categories.
Pie chart	GetClusters	Shows recipe distribution across cuisines.
Scatter plot	GetRecipes	Plots recipe protein against carbohydrates; point size reflects fat.
Diet filter	GetRecipes query + insights filter	Changes the selected diet and reloads recipe data.
Cuisine/search filters	Client-side filtered recipe data	Narrows chart points and table rows.
Refresh and CSV download	All endpoints / filtered recipes	Refreshes cloud data and exports filtered rows.

8. CORS and Authentication

The deployed functions use anonymous HTTP authorization, so the frontend does not require a function key. Because Azure Static Web Apps and Azure Functions use different origins, the exact frontend origin must be added to the Function App CORS allowed-origins list.

Required CORS value

After deployment, add the exact https://...azurestaticapps.net origin without a trailing slash. Use a specific origin rather than a wildcard in the final configuration.

Azure CLI alternative:

```
az functionapp cors add --resource-group YOUR_RESOURCE_GROUP --name diet-analysis-func01 --
allowed-origins https://YOUR-STATIC-APP.azurestaticapps.net
```

9. End-to-End Testing

1. Open the public Azure Static Web App URL and open browser Developer Tools.
2. Select Refresh data and confirm successful requests to all three Function endpoints.
3. Verify that all four charts, timing cards, and recipe rows display.
4. Change the diet filter and confirm the GetRecipes request includes ?diet_type=.
5. Change cuisine and search filters and confirm the scatter plot and table update.
6. Use Download CSV and verify the exported file contains only matching recipes.
7. Capture Network-tab evidence showing successful frontend-to-backend communication.
8. Run tests/integration_check.py as an additional endpoint smoke test.

Test case	Expected result	Evidence
All endpoints	JSON data loads with no browser errors.	Network tab and dashboard status pill.
Diet filter	Recipe endpoint is called with the selected diet.	Network request URL and changed rows.
Cuisine/search	Client-side data and charts update immediately.	Filtered table and scatter plot.
Execution metadata	Timing cards display milliseconds.	Dashboard screenshot.
Responsive layout	Dashboard remains readable on desktop and mobile.	Browser responsive mode screenshot.

10. Challenges and Solutions

Challenge	Solution implemented
Initial UI contained visual elements but incomplete live integration.	Added endpoint fetch logic, loading state, errors, filters, refresh, timing, and table rendering.
The three endpoints return different object shapes.	Created dedicated transformations for insights, recipes, and

Challenge	Solution implemented
	cuisine clusters.
Function execution metadata was not included in current array responses.	Measured client-side API duration and added support for future backend metadata.
Cross-origin requests may fail after the frontend receives a new domain.	Prepared exact portal and CLI CORS steps and surfaced a clear in-dashboard error message.
Framework build configuration can delay a time-sensitive submission.	Prepared a plain static deployment with no npm build dependency.
Multiple branches require final assembly.	Prepared a manya-integration merge workflow and pull-request checklist.

11. Cloud Practices and Security

- AzureWebJobsStorage is read from environment variables rather than hard-coded in source code.
- local.settings.json and connection strings are excluded from GitHub.
- The frontend calls the Function API and does not expose the Storage Account connection string.
- The final CORS policy should contain the exact Static Web App origin rather than a wildcard.
- Static Web Apps security headers are defined in staticwebapp.config.json.
- Dataset processing occurs in the Function App, not inside the browser.
- Screenshots must hide secrets, access keys, and full connection strings.

12. GitHub Repository Assembly

The final main branch should contain the deployed Function source, the selected frontend folder, documentation files, deployment instructions, tests, and the Azure Static Web Apps workflow generated during deployment.

9. Create a manya-integration branch from main.
10. Merge origin/anagha-backend and origin/jasmeen-frontend.
11. Add the completed Member C frontend, documentation, deployment, and test files.
12. Review git status and verify that no local.settings.json or connection string is included.
13. Push the integration branch and merge it into main through a pull request.
14. Deploy Azure Static Web Apps from the final main branch.

13. Required Screenshot Evidence

Replace the boxes below with the team screenshots before exporting the final submission PDF. The editable DOCX is included so this can be completed quickly.

Figure 2. Azure resources and live Blob dataset

INSERT SCREENSHOT HERE

Show the resource group containing the Function App, Storage Account, and Static Web App. Include a second image or crop showing datasets/All_Diets.csv.

Evidence note: Make sure the resource name, URL, or successful response is visible. Hide connection strings and secrets.

Figure 3. Three live Azure Function endpoints

INSERT SCREENSHOT HERE

Show GetNutritionalInsights, GetRecipes, and GetClusters returning JSON in a browser or Postman.

Evidence note: Make sure the resource name, URL, or successful response is visible. Hide connection strings and secrets.

Figure 4. Function App CORS configuration

INSERT SCREENSHOT HERE

Show the exact Azure Static Web App URL in the Function App Allowed Origins list.

Evidence note: Make sure the resource name, URL, or successful response is visible. Hide connection strings and secrets.

Figure 5. Public dashboard

INSERT SCREENSHOT HERE

Show the deployed Azure Static Web App with timing cards, filters, and all four visualizations populated.

Evidence note: Make sure the resource name, URL, or successful response is visible. Hide connection strings and secrets.

Figure 6. End-to-end browser Network test

INSERT SCREENSHOT HERE

Show successful requests from the Static Web App to all three Function endpoints.

Evidence note: Make sure the resource name, URL, or successful response is visible. Hide connection strings and secrets.

Figure 7. GitHub final main branch

INSERT SCREENSHOT HERE

Show the final repository structure and successful Azure Static Web Apps workflow.

Evidence note: Make sure the resource name, URL, or successful response is visible. Hide connection strings and secrets.

14. Deployment and Submission Checklist

- GitHub collaborator invitation accepted.
- Backend and frontend branches merged into the integration branch.
- Final files merged into main.
- Azure Static Web App deployed from main.
- Exact Static Web App origin added to Function App CORS.
- Public dashboard tested with all three live endpoints.
- Public frontend URL pasted on the cover page and deliverables section.
- All screenshot placeholders replaced.
- DOCX exported to PDF and reviewed for clipping or missing images.
- Submission includes deployed Function URL, frontend URL, GitHub repository, and documentation PDF.

15. Deliverables

Deliverable	Final value / location
Azure Function URL	https://diet-analysis-func01-d7fdd4awc3dacqew.canadacentral-01.azurewebsites.net
Azure Static Web App URL	[PASTE AFTER DEPLOYMENT]
GitHub repository	https://github.com/anagha-04-roy/diet-analysis-phase2
Documentation PDF	docs/Project_Phase2_Cloud_Dashboard_Documentation.pdf
Architecture diagram	docs/architecture-diagram.png
Integration test	tests/integration_check.py

16. Conclusion

The Phase 2 solution demonstrates the complete transition from a local prototype to a cloud-connected application. Azure Blob Storage provides the live dataset, Azure Functions perform reusable serverless analysis, and Azure Static Web Apps delivers an interactive dashboard. The final integration provides meaningful visualizations, user controls, execution metadata, error handling, test evidence, and clear documentation of the architecture and workflow.

References

- Microsoft Learn. Configure function app settings in Azure Functions - CORS.
- Microsoft Learn. Azure CLI: az functionapp cors.
- Microsoft Learn. Quickstart: Build your first static web app.
- GitHub repository: https://github.com/anagha-04-roy/diet-analysis-phase2
- Project brief: Project Phase 2 - Cloud Dashboard Development.